JAVA DSA
SOLVED ALGORITHMS

VOLUME 2 — LINKED LISTS • STACKS • QUEUES • DEQUES • HEAPS

Rules • Pattern recognition • Java code • Dry runs • Complexity • Interview problem bank

A detailed continuation of Volume 1. This volume covers the major algorithms in these data-structure groups; it does not claim to contain every question ever asked in interviews.

How to solve an interview algorithm

• Recognise the input structure and the repeated operation.

• State the invariant: what remains true after every iteration.

• Explain why the chosen structure improves brute force.

• Dry-run a normal case and an edge case before coding.

• Test null/empty, one element, duplicates, boundaries and overflow.

• State time and auxiliary space complexity and justify both.

1. Linked Lists

Definition: A linked list is a chain of nodes. Each node stores data and a reference to the next node.

Node template and rules

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int v) { val = v; }
    ListNode(int v, ListNode n) { val = v; next = n; }
}
```

• Save next before changing a pointer; otherwise the remaining list can be lost.

• Use a dummy node when the head may be removed or replaced.

• Compare node references with == for cycle/intersection problems.

• Fast/slow pointers find a midpoint or detect cycles without extra memory.

Reverse linked list — pointer reversal

Invariant: prev is the reversed prefix; curr is the first node of the untouched suffix.

```
ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next;
        curr.next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
```

Dry run: 1→2→3→null. Iteration 1 makes 1→null; iteration 2 makes 2→1; iteration 3 makes 3→2→1. Return prev. **Time:** O(n), **space:** O(1).

Merge two sorted lists — dummy node

```
ListNode mergeTwoLists(ListNode a, ListNode b) {
    ListNode dummy = new ListNode(0), tail = dummy;
    while (a != null && b != null) {
        if (a.val <= b.val) { tail.next = a; a = a.next; }
        else { tail.next = b; b = b.next; }
        tail = tail.next;
    }
    tail.next = (a != null) ? a : b;
    return dummy.next;
}
```

Dry run: 1→4 and 2→3: attach 1, 2, 3, then remaining 4. O(n+m) time, O(1) extra space.

Cycle detection — Floyd's algorithm

```
boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}
```

Slow moves one link, fast moves two. In a cycle, fast eventually catches slow. O(n) time, O(1) space.

Middle node — fast/slow

```
ListNode middleNode(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow; // second middle for even length
}
```

O(n) time, O(1) space.

Remove Nth node from end — two pointers + dummy

```
ListNode removeNthFromEnd(ListNode head, int n) {
    ListNode dummy = new ListNode(0, head);
    ListNode fast = dummy, slow = dummy;
    for (int i = 0; i < n; i++) fast = fast.next;
    while (fast.next != null) {
        fast = fast.next;
        slow = slow.next;
    }
    slow.next = slow.next.next;
    return dummy.next;
}
```

Fast stays n nodes ahead. When fast reaches the last node, slow is just before the target. Assumption: 1 ≤ n ≤ length. O(n) time, O(1) space.

Palindrome linked list — middle + reverse

```
boolean isPalindrome(ListNode head) {
    if (head == null || head.next == null) return true;
    ListNode slow = head, fast = head;
    while (fast.next != null && fast.next.next != null) {
        slow = slow.next; fast = fast.next.next;
    }
    ListNode right = reverseList(slow.next);
    ListNode a = head, b = right;
    boolean ok = true;
    while (b != null) {
        if (a.val != b.val) { ok = false; break; }
        a = a.next; b = b.next;
    }
    slow.next = reverseList(right); // restore original list
    return ok;
}
```

O(n) time, O(1) extra space. Reversing back is useful if the interviewer expects the input list to remain unchanged.

Other linked-list algorithms — rules

• Reorder list: find middle → reverse second half → weave alternating nodes.

• Add two numbers: add digits with carry; continue while either list or carry remains.

• Intersection: pointer switching; each pointer traverses both lists, aligning path lengths.

• Reverse k-group: verify a complete group exists, reverse it, then reconnect boundaries.

• Merge k lists: min-heap with one current node per list; O(N log k).

• Copy random-pointer list: HashMap old→new, or interweave copies for O(1) auxiliary space.

• LRU cache: HashMap + doubly linked list; get/put O(1).

2. Stack

Definition: LIFO — last in, first out. In Java, use ArrayDeque for stack operations.

Valid parentheses

```
boolean isValid(String s) {
    Deque<Character> st = new ArrayDeque<>();
    for (char ch : s.toCharArray()) {
        if (ch == '(' || ch == '[' || ch == '{') st.push(ch);
        else {
            if (st.isEmpty()) return false;
            char op = st.pop();
            if ((ch == ')' && op != '(') ||
                (ch == ']' && op != '[') ||
                (ch == '}' && op != '{')) return false;
        }
```

```
    }
    return st.isEmpty();
}
```

Rule: each closer must match the most recent unmatched opener. O(n) time, O(n) space.

Min Stack — maintain current minimum

```
class MinStack {
    Deque<Integer> values = new ArrayDeque<>();
    Deque<Integer> mins = new ArrayDeque<>();
    void push(int x) {
        values.push(x);
        mins.push(mins.isEmpty() ? x : Math.min(x, mins.peek()));
    }
    void pop() { values.pop(); mins.pop(); }
    int top() { return values.peek(); }
    int getMin() { return mins.peek(); }
}
```

Every operation O(1); O(n) storage.

Evaluate Reverse Polish Notation

```
int evalRPN(String[] tokens) {
    Deque<Integer> st = new ArrayDeque<>();
    for (String t : tokens) {
        if (t.equals("+") || t.equals("-") ||
            t.equals("*") || t.equals("/")) {
            int b = st.pop(), a = st.pop(); // order matters
            switch (t) {
                case "+": st.push(a + b); break;
                case "-": st.push(a - b); break;
                case "*": st.push(a * b); break;
                default: st.push(a / b);
            }
        } else st.push(Integer.parseInt(t));
    }
    return st.pop();
}
```

For subtraction/division, pop b first and a second, then compute a op b. O(n) time, O(n) space.

Monotonic stack — next greater element

Keep indices with values in decreasing order. When current value is larger, it resolves the next-greater answer for popped indices.

```
int[] nextGreater(int[] a) {
    int[] ans = new int[a.length];
    Arrays.fill(ans, -1);
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < a.length; i++) {
        while (!st.isEmpty() && a[i] > a[st.peek()])
            ans[st.pop()] = a[i];
        st.push(i);
    }
    return ans;
}
```

Each index is pushed/popped once: O(n) time, O(n) space.

Daily temperatures

```
int[] dailyTemperatures(int[] t) {
    int[] ans = new int[t.length];
    Deque<Integer> st = new ArrayDeque<>();
    for (int i = 0; i < t.length; i++) {
        while (!st.isEmpty() && t[i] > t[st.peek()]) {
            int j = st.pop();
            ans[j] = i - j;
        }
        st.push(i);
    }
    return ans;
}
```

Dry run [73,74,75,71]: 74 resolves day 0 with wait 1; 75 resolves day 1 with wait 1; unresolved days remain 0. O(n) time.

Largest rectangle in histogram

```
int largestRectangleArea(int[] h) {
    Deque<Integer> st = new ArrayDeque<>();
    int best = 0;
    for (int i = 0; i <= h.length; i++) {
        int cur = (i == h.length) ? 0 : h[i];
        while (!st.isEmpty() && h[st.peek()] > cur) {
```

```
            int height = h[st.pop()];
            int left = st.isEmpty() ? -1 : st.peek();
            int width = i - left - 1;
            best = Math.max(best, height * width);
        }
        st.push(i);
    }
    return best;
}
```

Increasing stack finds the first smaller boundary on both sides. Sentinel height 0 flushes remaining bars. O(n) time, O(n) space.

Stack pattern rules

• Next smaller: reverse the comparison in the monotonic stack.

• Circular next greater: scan 2n positions using i % n; push indices only during first pass.

• Stock span: pop previous prices ≤ current; span ends after previous greater price.

• Remove K digits: increasing digit stack; remove larger previous digits while k remains.

• Subarray minimum contribution: value × number of valid left choices × right choices; use asymmetric strictness for duplicates.

• Trapping rain water: stack of indices; popped bottom forms a basin bounded by current right wall and new stack top.

3. Queue and Deque

Definition: Queue is FIFO. Deque allows insertion/removal at both ends. Java ArrayDeque supports both.

Queue using two stacks

```
class MyQueue {
    Deque<Integer> in = new ArrayDeque<>();
    Deque<Integer> out = new ArrayDeque<>();
    void push(int x) { in.push(x); }
    int pop() { move(); return out.pop(); }
    int peek() { move(); return out.peek(); }
    boolean empty() { return in.isEmpty() && out.isEmpty(); }
    void move() {
        if (out.isEmpty())
            while (!in.isEmpty()) out.push(in.pop());
    }
}
```

Transfer only when output stack is empty. Each element moves at most once: O(1) amortized per operation.

Sliding window maximum — monotonic deque

```
int[] maxSlidingWindow(int[] a, int k) {
    int n = a.length;
    int[] ans = new int[n - k + 1];
    Deque<Integer> dq = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        while (!dq.isEmpty() && dq.peekFirst() <= i - k)
            dq.pollFirst();
        while (!dq.isEmpty() && a[dq.peekLast()] <= a[i])
            dq.pollLast();
        dq.offerLast(i);
        if (i >= k - 1) ans[i - k + 1] = a[dq.peekFirst()];
    }
    return ans;
}
```

Deque stores indices in decreasing value order. Expired indices leave front; dominated values leave back. O(n) time, O(k) space.

Circular queue — array + modulo

```
class MyCircularQueue {
    int[] a; int front = 0, size = 0;
    MyCircularQueue(int k) { a = new int[k]; }
    boolean enQueue(int v) {
        if (size == a.length) return false;
        a[(front + size) % a.length] = v; size++;
        return true;
    }
    boolean deQueue() {
        if (size == 0) return false;
        front = (front + 1) % a.length; size--;
        return true;
    }
    int Front() { return size == 0 ? -1 : a[front]; }
    int Rear() {
```

```
        return size == 0 ? -1 : a[(front + size - 1) % a.length];
    }
}
```

Empty iff size=0; full iff size=capacity. All operations O(1). Handle capacity 0 separately if allowed by constraints.

4. Heaps and Priority Queues

Definition: A heap is a complete binary tree. Min-heap root is smallest; max-heap root is largest. Java PriorityQueue is min-heap by default.

• Top K largest → min-heap of size K.

• Top K smallest → max-heap of size K.

• Merge K sorted lists/streams → min-heap with one current item per list.

• Running median → max-heap lower half + min-heap upper half.

• Scheduling by earliest finishing time → min-heap of end times.

Kth largest — bounded min-heap

```
int findKthLargest(int[] nums, int k) {
    PriorityQueue<Integer> pq = new PriorityQueue<>();
    for (int x : nums) {
        pq.offer(x);
        if (pq.size() > k) pq.poll();
    }
    return pq.peek();
}
```

Invariant: heap stores the k largest values seen; root is the kth largest. O(n log k) time, O(k) space.

Top K frequent elements

```
int[] topKFrequent(int[] nums, int k) {
    Map<Integer,Integer> f = new HashMap<>();
    for (int x : nums) f.put(x, f.getOrDefault(x, 0) + 1);
    PriorityQueue<Integer> pq =
        new PriorityQueue<>((a,b) -> Integer.compare(f.get(a), f.get(b)));
    for (int x : f.keySet()) {
        pq.offer(x);
        if (pq.size() > k) pq.poll();
    }
    int[] ans = new int[k];
    for (int i = k - 1; i >= 0; i--) ans[i] = pq.poll();
    return ans;
}
```

Let m be distinct values. O(n + m log k + k log k) time, O(m+k) space. Use Integer.compare, not subtraction, in comparators.

Merge K sorted lists — min-heap

```
ListNode mergeKLists(ListNode[] lists) {
    PriorityQueue<ListNode> pq =
        new PriorityQueue<>((x,y) -> Integer.compare(x.val, y.val));
    for (ListNode node : lists) if (node != null) pq.offer(node);
    ListNode dummy = new ListNode(0), tail = dummy;
    while (!pq.isEmpty()) {
        ListNode node = pq.poll();
        tail.next = node; tail = node;
        if (node.next != null) pq.offer(node.next);
    }
    tail.next = null;
    return dummy.next;
}
```

N total nodes, k lists: O(N log k) time, O(k) heap space.

Median from data stream — two heaps

```
class MedianFinder {
    PriorityQueue<Integer> low =
        new PriorityQueue<>(Collections.reverseOrder());
    PriorityQueue<Integer> high = new PriorityQueue<>();
    void addNum(int x) {
        low.offer(x);
        high.offer(low.poll());
        if (low.size() < high.size()) low.offer(high.poll());
    }
    double findMedian() {
        if (low.size() > high.size()) return low.peek();
        return ((long) low.peek() + high.peek()) / 2.0;
    }
```

```
}
```

Invariant: low contains lower half, high upper half; sizes differ by at most one. add O(log n), median O(1). Cast to long before addition.

More heap patterns

• K closest points: max-heap size K by squared distance.

• Meeting Rooms II: sort by start; min-heap of end times; release rooms that have ended.

• Task Scheduler: max-heap of frequencies plus cooldown queue, or derive the frequency formula.

• Reorganize String: repeatedly take two most frequent remaining characters.

• Furthest Building: use ladders for largest climbs; min-heap tracks climbs paid with bricks.

• Find K pairs with smallest sums: heap over the frontier of candidate index pairs.

5. Interview practice bank — from your uploaded PDFs

The uploaded banks list these problems/patterns. Use them as the required practice set for this volume; the entries below are problem prompts, not claims that every platform has identical wording.

Linked Lists

• Reverse Linked List

• Reverse Linked List II

• Merge Two Sorted Lists

• Merge k Sorted Lists

• Linked List Cycle

• Linked List Cycle II

• Middle of the Linked List

• Remove Nth Node From End

• Reorder List

• Palindrome Linked List

• Add Two Numbers

• Add Two Numbers II

• Intersection of Two Linked Lists

• Copy List with Random Pointer

• Rotate List

• Partition List

• Swap Nodes in Pairs

• Reverse Nodes in k-Group

• Remove Duplicates from Sorted List II

• Flatten a Multilevel Doubly Linked List

• LRU Cache

• Design Browser History

Stack / Queue / Deque

• Valid Parentheses

• Min Stack

• Implement Stack using Queues

• Implement Queue using Stacks

- Evaluate Reverse Polish Notation
- Daily Temperatures
- Next Greater Element I
- Next Greater Element II
- Online Stock Span
- Largest Rectangle in Histogram
- Maximal Rectangle
- Sliding Window Maximum
- Asteroid Collision
- Decode String
- Remove K Digits
- 132 Pattern
- Sum of Subarray Minimums
- Sum of Subarray Ranges
- Trapping Rain Water
- Basic Calculator
- Basic Calculator II
- Simplify Path
- Car Fleet
- Design Circular Queue
- Design Front Middle Back Queue

Heaps / Priority Queues

- Kth Largest Element in an Array
- Kth Largest Element in a Stream
- Top K Frequent Elements
- Top K Frequent Words
- K Closest Points to Origin
- Find Median from Data Stream
- Merge k Sorted Lists
- Smallest Range Covering Elements from K Lists
- Find K Pairs with Smallest Sums
- Task Scheduler
- Meeting Rooms II
- Reorganize String
- Furthest Building You Can Reach
- IPO
- Seat Reservation Manager
- Process Tasks Using Servers
- Single-Threaded CPU

6. Complexity quick reference

Algorithm	Time	Extra space
Reverse / middle / cycle list	O(n)	O(1)
Merge two lists	O(n+m)	O(1)
Stack push/pop/peek	O(1)	O(n) storage
Monotonic stack/deque scan	O(n) amortized	O(n)
Queue via two stacks	O(1) amortized	O(n)
Heap offer/poll/peek	O(log n) / O(1)	O(n)
Top K bounded heap	O(n log k)	O(k)
Merge K lists	O(N log k)	O(k)
Median stream add/query	O(log n) / O(1)	O(n)

7. Java interview checklist

• Use ArrayDeque for stack/queue; it does not accept null.

• PriorityQueue is a min-heap by default; use Collections.reverseOrder() for max-heap.

• Use Integer.compare(a,b), not a-b, in comparators.

• Monotonic stack: state increasing/decreasing and whether it stores values or indices.

• Heap: explain what the root represents and why the heap size is bounded.

• Linked list: draw pointer changes; preserve the untouched suffix before rewiring.

• Test empty, one node, two nodes, duplicates, sorted/reverse-sorted input and k=1/k=n.

Completion standard: Recognise the pattern in an unfamiliar statement, explain the invariant, derive the Java code, test edge cases and justify complexity without notes.